

COMP6210 - Big Data Assignment 1

MapReduce

Semester 2, 2026

School of Computing, Macquarie University

Datasets:

The FIFA 2026 player performance dataset "FIFA2026_Player.zip" is available on iLearn. The dataset contains player match records from FIFA World Cup 2026, league matches, Champions League, domestic cup matches, and World Cup qualifiers. The zip file includes (1) "data.csv", which stores the player performance records, and (2) "field_descriptors.txt", which provides field definitions and detailed descriptions.

Programming Environments:

- **MongoDB:** Provides database services for managing data.
- **Python:** The primary programming language used to complete the assignment.
 - **PyMongo:** Used to connect Python to MongoDB and perform database operations.
 - **MrJob:** Used to implement MapReduce programs in Python.

Task 1: Data Curation (25 marks)

- **Task 1.1 - Data Import, Cleaning, and Extraction (15 marks):**

Step 1: Import the dataset.

Use Python to connect to MongoDB, create a database named "Assignment_1", and import the dataset into a collection named "FIFA_players".

Step 2: Clean the data and store the results.

Apply the following cleaning rules **in the specified order**. Once a record is removed in a cleaning step, it must not be used in any subsequent cleaning, inference, or calculation. Store all records remaining after the cleaning process in a new MongoDB collection named "FIFA_players_cleaned".

1. **String standardization:** For all string fields, remove leading and trailing spaces and replace multiple consecutive internal spaces with a single space.
2. **Competition standardization:** Standardise all competition names to one of the following five values: "FIFA World Cup 2026", "Champions League", "Domestic Cup", "World Cup Qualifiers", or "League".
3. **Missing identifiers:** After string standardisation, remove any record with a missing value in one or more required identifier fields: `player_name`, `competition`, or `match_id`.
4. **Invalid numeric values:** Remove any record containing a negative value in any of the following fields: `market_value_eur_m`, `minutes_played`, `goals`, `assists`, `shots_on_target`, `tackles`, `match_rating`, or `fouls_committed`.
5. **Unknown nationality:** For each record where `nationality` = "Unknown", use records with the same `player_name`, `age`, and `club` to infer the nationality. If the known nationalities in all matching records are the same, replace "Unknown" with that nationality; otherwise, retain "Unknown".
6. **Missing market value:** For records with a missing `market_value_eur_m`, replace the missing value with the average `market_value_eur_m` calculated from records with the same `competition` and `player_name`.

If no non-missing value is available for that combination, retain the value as NA.

Step 3: For each cleaned document in "FIFA_players_cleaned", extract the following fields:

	√ (1) {_id: 6a7d4c75c01da07738225397}	{ 22 fields }
• player name	id	6a7d4c75c01da07738225397
• nationality	player_name	Cristian Romero
• club	nationality	Argentina
• position	club	Tottenham Hotspur
• market value	position	CB
• competition	age	32
• match_id	market_value_eur_m	3.7
• minutes_played	season	2025/26
• goals	competition	Champions League
• assists	match_id	108728
• match_rating	minutes_played	90
	goals	0
	assists	0
	shots_on_target	0
	pass_accuracy_pct	61.9
	tackles	4
	distance_covered_km	8.83
	match_rating	6.5
	yellow_card	true
	red_card	false
	fouls_committed	0
	injured_flag	false

❖ Output Requirements:

For Task 1.1, submit the following two output files:

1. "player_data.txt"

Write the fields extracted in Step 3 to "player_data.txt" in the specified order:

player_name, nationality, club, position, market_value_eur_m, competition, match_id, minutes_played, goals, assists, match_rating

Each record must occupy a single line, with the fields separated using a consistent delimiter (e.g., comma or tab).

Refer to the provided screenshot for formatting guidelines (note that the screenshot is for formatting reference only; actual results may vary).

Alisson Becker,Brazil,Liverpool,GK,9.59,World Cup Qualifiers,112525,90,0,0,7.1
Mohammed Kudus,Ghana,Tottenham Hotspur,RW,20.22,League,101838,21,0,0,4.2
Karim Benzema,France,Al Ittihad,ST,0.96,World Cup Qualifiers,112759,90,0,0,6.8
Rodri,Spain,Manchester City,CDM,31.27,League,109500,83,0,0,6.4
Bukayo Saka Jr,England,Arsenal,RW,19.25,League,101607,71,0,1,7.0
Vitor Roque,Brazil,Palmeiras,ST,50.06,Champions League,107971,43,0,0,5.6
Mohamed Salah,Egypt,Liverpool,RW,10.69,Champions League,111734,86,1,1,7.8

2. "cleaning_report.txt"

Summarize the data-cleaning process by reporting:

- the total number of records before cleaning;
- the total number of records after cleaning;
- the number of records with nationality = "Unknown" before cleaning;
- the number of records with missing market_value_eur_m before cleaning;
- the number of records whose string fields were standardized.

- **Task 1.2 - Data Organization (10 marks):** Using the generated "player_data.txt" file as input, implement a **MapReduce program** to organize the records by competition. Sort the competitions in ascending alphabetical order. Within each competition, sort the records first by player_name and then by match_id, both in ascending order. Partial results are similar to the following screenshot (note that the screenshot is for formatting reference only; actual results may vary).



Champions League, Diogo Dalot, 106313, Portugal, Manchester United, RB, 7.60
 Champions League, Erling Braut Haaland, 111388, Norway, Manchester City, ST, 6.40
 Champions League, Jack Grealish, 103000, England, Everton, LW, 4.20
 Champions League, Randal Kolo, 110157, France, Juventus, ST, 5.20
 Domestic Cup, Harry Kane, 108167, England, Bayern Munich, ST, 6.00
 Domestic Cup, Nick Pope, 112882, England, Newcastle United, GK, 3.60
 Domestic Cup, Steven Bergwijn, 111200, Netherlands, Ajax, LW, 7.00
 League, Eduardo Camavinga, 102076, France, Real Madrid, CM, 6.60
 League, Jurrien Timber, 104367, Netherlands, Arsenal, RB, 3.70
 League, Kobbie Mainoo, 107623, England, Manchester United, CM, 7.00
 League, Marcus Rashford, 105425, England, Barcelona, LW, 4.60

ascending order

Task 2: Data Analysis with MapReduce (45 marks)

Using the generated "player_data.txt" file as input, implement **three MapReduce programs** to complete the following analysis tasks Task 2.1, Task 2.2 and Task 2.3. In each MapReduce program, you may use multiple MapReduce Steps to complete the task. For Tasks 2.1 and 2.3, use the following metrics to evaluate player performance. For each record, calculate:

$\text{performance_points} = \text{goals} * 4 + \text{assists} * 3 + \text{match_rating}$

For a given set of records, calculate:

$\text{total_performance_points} = \text{sum of performance_points over all relevant records}$

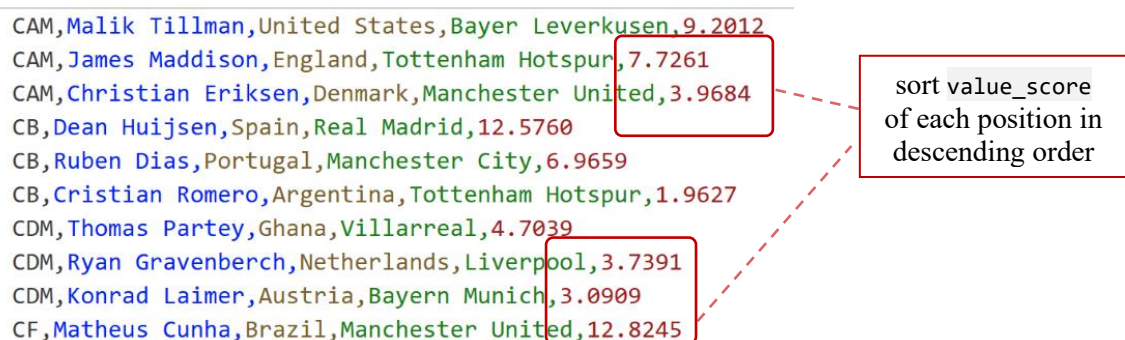
$\text{total_minutes} = \text{sum of minutes_played over all relevant records}$

$\text{performance_per90} = \text{total_performance_points} / \text{total_minutes} * 90$

Round calculated decimal values to **four decimal places** in the output.

- **Task 2.1 - Value-for-Money Players by Position (15 marks):** Use only records where competition = "FIFA World Cup 2026". For each player, calculate performance_per90 using the formula defined above. Calculate avg_market_value as the average of all non-NA market_value_eur_m values from the player's World Cup records, and then calculate value_score = performance_per90 / avg_market_value. Only include players with total_minutes >= 180 and a valid avg_market_value > 0.

For each position, identify the **top three players** with the highest value_score. Sort players within each position by value_score in descending order. If multiple players have the same value_score, break ties by sorting player_name in ascending alphabetical order. The output should include position, player_name, nationality, club, and value_score. Partial results are similar to the following screenshot (note that the



CAM, Malik Tillman, United States, Bayer Leverkusen, 9.2012
 CAM, James Maddison, England, Tottenham Hotspur, 7.7261
 CAM, Christian Eriksen, Denmark, Manchester United, 3.9684
 CB, Dean Huijsen, Spain, Real Madrid, 12.5760
 CB, Ruben Dias, Portugal, Manchester City, 6.9659
 CB, Cristian Romero, Argentina, Tottenham Hotspur, 1.9627
 CDM, Thomas Partey, Ghana, Villarreal, 4.7039
 CDM, Ryan Gravenberch, Netherlands, Liverpool, 3.7391
 CDM, Konrad Laimer, Austria, Bayern Munich, 3.0909
 CF, Matheus Cunha, Brazil, Manchester United, 12.8245

sort value_score of each position in descending order

screenshot is for formatting reference only; actual results may vary).

- **Task 2.2 - Club Contribution Analysis (15 marks):** Use records where `competition = "FIFA World Cup 2026"`. For each player within each club, calculate `total_goals`, `total_assists`, and `attack_score = total_goals + total_assists`. Then aggregate the player-level results for each club to obtain `total_players` and `total_attack_score`. where `total_players` is the number of distinct players in the club and `total_attack_score` is the sum of the individual `attack_score` values of all players in that club.

For each club, define `top_player` as the player with the highest individual `attack_score`, and `top_player_score` as that player's `attack_score`. If multiple players have the same highest `attack_score`, select the player whose `player_name` comes first in ascending alphabetical order.

Identify the **top 10 clubs** with the highest `total_attack_score`. Sort the clubs by `total_attack_score` in descending order. If multiple clubs have the same `total_attack_score`, break ties by sorting `club` in ascending alphabetical order. The output should include `club`, `total_players`, `total_attack_score`, `top_player`, and `top_player_score`. Partial results are similar to the following screenshot (note that the screenshot is for formatting reference only; actual results may vary).

Arsenal	18	30	Kai Havertz	8
Liverpool	11	25	Federico Chiesa	8
Bayern Munich	12	24	Serge Gnabry	5
Manchester United	17	22	Matheus Cunha	6
Inter Milan	9	20	Marcus Thuram	6
Chelsea	10	14	Alejandro Garnacho	5
Real Madrid	16	14	Rodrygo	4
Barcelona	12	12	Dani Olmo	4
Atletico Madrid	9	11	Antoine Griezmann	3
Bayer Leverkusen	4	11	Victor Boniface	6

total_attack_score,
descending order

- **Task 2.3 - World Cup vs Club Performance Gap (15 marks):** Compare each player's performance in the FIFA World Cup 2026 with their performance in club competitions, including League, Champions League, and Domestic Cup. For each player, calculate `worldcup_per90` using all FIFA World Cup 2026 records and `club_per90` using all records from the three club competitions combined, following the `performance_per90` formula defined above. Then calculate `performance_gap = worldcup_per90 - club_per90`. Only include players with `worldcup_minutes >= 180` and `club_minutes >= 180`. For each player, `club` is consistent across the records used in this task.

Identify the **top 10 players** with the highest `performance_gap`. Sort the players by `performance_gap` in descending order. If multiple players have the same `performance_gap`, break ties by sorting `player_name` in ascending alphabetical order. The output should include `player_name`, `nationality`, `club`, and `performance_gap`. Partial results are similar to the following screenshot (note that the screenshot is for formatting reference only; actual results may vary).

Matheus Cunha	Brazil	Manchester United	5.99
Randal Kolo	France	Juventus	5.31
Kai Havertz	Germany	Arsenal	4.91
Dusan Vlahovic	Serbia	Juventus	3.43
Dominik Szoboszlai	Hungary	Liverpool	2.40
Erling Haaland	Norway	Manchester City	2.08
Malik Tillman	United States	Bayer Leverkusen	-1.92
Alexander Isak	Sweden	Liverpool	-2.78
Aleksandar Pavlovic	Germany	Bayern Munich	-3.71
Mohamed Salah	Egypt	Liverpool	-4.70

performance_gap,
descending order

Task 3: MapReduce Flowcharts (30 marks)

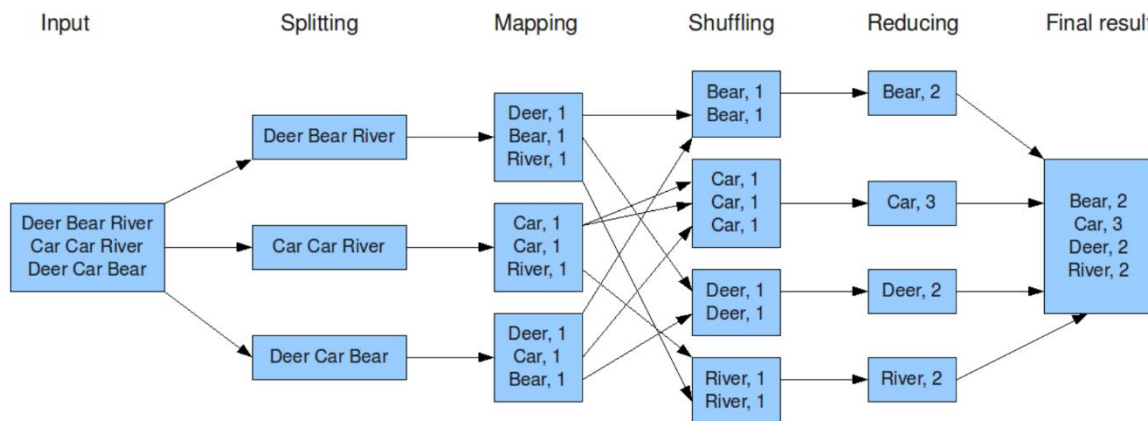
For the **three MapReduce programs** in Task 2 (Tasks 2.1, 2.2, and 2.3), create a Word or PDF document containing **three separate flowcharts** - one for **each program** - illustrating the complete MapReduce process.

❖ Requirements:

Each flowchart must be based on concrete examples from the generated "player_data.txt" file and illustrate the complete MapReduce process, including:

1. **Input** - sample input records;
2. **Splitting** - how the input is divided;
3. **Mapping** - the key-value pairs generated by the Mapper;
4. **Shuffling** - how intermediate values are grouped by key;
5. **Reducing** - the aggregation or processing performed by the Reducer;
6. **Output** - the resulting output records.

Use sufficient concrete records to clearly demonstrate how multiple input records are mapped, grouped, and aggregated. Show the actual input and output at each stage, with arrows indicating the data flow. If your implementation uses multiple MapReduce steps, each step must be clearly represented in the flowchart.



❖ Guidelines:

- Follow the given example: use multiple **concrete data examples** (e.g., "Deer Bear River" → word counts) rather than abstract placeholders.
- Show actual input and output at each step, with arrows indicating data flow.
- Ensure that the flowcharts accurately reflect your Task 2 implementations.

❖ Diagram Creation Recommendation:

We recommend using the online diagramming tool <https://miro.com/> to create the three flowcharts for Task 3. If you use Miro, you may choose to submit only the sharing link(s) to your flowcharts in the Word or PDF document. Ensure that all submitted links are accessible to the teaching staff without requiring additional permission.

Programming Environments:

You **must** use **Python** to complete all tasks of this assignment. The use of other programming languages is strictly prohibited and will result in **zero marks**. For Task 1.2 and Tasks 2.1–2.3, all required data processing must be implemented entirely using MapReduce. Only **mapper**, **combiner**, **reducer**, and **steps** functions may be used within the MapReduce workflow. Other MapReduce components or additional Python scripts outside the MapReduce workflow are not permitted. Failure to follow this requirement will result in zero marks for the

corresponding task, even if the final output is correct.



Submission:

Submit a single zip file named:

FirstName_LastName_Assignment1.zip

via iLearn. The zip file should include:

- **Task 1 source code:** *task1_1.py*, *task1_2.py*
- **Task 2 source code:** *task2_1.py*, *task2_2.py*, *task2_3.py*
- **Task 1 output files:** *cleaning_report.txt*, *player_data.txt*, *output1_2.txt*
- **Task 2 output files:** *output2_1.txt*, *output2_2.txt*, *output2_3.txt*
- **Task 3 flowchart document:** *flowcharts.docx* or *flowcharts.pdf*